

Reflective Analysis of Programming Paradigms and Secure Software Development

Student ID: 5653630, December 2025

Programming Languages for Cyber Security (WM245), WMG, University of Warwick

Contents

1	Introduction	1
2	Programming Paradigms	1
2.1	Object Oriented Programming (OOP)	1
2.1.1	Encapsulation	1
2.1.2	Inheritance	2
2.1.3	Polymorphism	2
2.2	OOP Security Consideration	3
2.3	Advantages of OOP (Java)	4
2.4	Limitations of OOP (Java)	4
2.5	Alternative Implementation to OOP	4
2.6	Event-Driven Programming (EDP)	6
2.6.1	Event Loop	6
2.6.2	Event Sources	6
2.6.3	Event Listeners	7
2.6.4	Event Handlers	7
2.7	EDP Security Consideration	7
2.8	Advantages & limitations of EDP (Java)	8
2.9	Alternative Implementation to EDP	8
2.10	Functional Programming (FP)	9
2.11	FP Security Consideration	9
2.12	Advantages & limitations of FP (Java)	9
2.13	Alternative Implementation of FP (Haskell)	10
3	Reflection	10
3.1	Revised OOP Solution	10
3.1.1	Revised Encapsulation	11
3.1.2	Revised Inheritance	11
3.1.3	Revised Polymorphism	11
3.2	Revised EDP Solution	13

3.2.1	Revised Event Loop	13
3.2.2	Revised Event Sources	13
3.2.3	Revised Event Listeners	14
3.3	Revised Future FP Solution	14
4	Conclusion	14
	References	15
	Appendices	16
A	Implementation of OOP Principles	16
B	Implementation of EDP Principles	18
C	Implementation of FP Principles	20
D	Revised UIS class hierarchy	22
E	Revised event-driven implementation	24
F	Code Base Access	28

1 Introduction

This report serves to outline my progression through Warwick's PLCS module. It evaluates my understanding of choosing a particular programming language or paradigm over another when implementing a solution. It demonstrates my growth in considering trade-offs between approaches by comparing my initial and revised implementations, particularly when balancing performance and security.

2 Programming Paradigms

2.1 Object Oriented Programming (OOP)

OOP refers to a paradigm that organises software design around data, or objects, rather than functions and logic. Its main aim is to use programming languages such as Python, Java and C++ to emulate real-world entities (objects) through its core principles [1].

During my own practice, I wanted to improve my understanding of OOP by developing a University Information System (UIS) with a suitable paradigm and language, with Java being the final choice. This is because Java is considered a naturally object-oriented language that supports OOP's core principles using techniques such as encapsulation, inheritance and polymorphism [2]. These principles were then included in my early solution (Appendix A), as shown below:

2.1.1 Encapsulation

```
1 class WarwickPerson {
2
3     protected String name;
4     protected String universityId;
5
6     public WarwickPerson(String name, String universityId) {
7         this.name = name;
8         this.universityId = universityId;
9     }
10
11     public String getName() {
12         return name;
13     }
14
15     public String getUniversityId() {
16         return universityId;
17     }
18 }
```

Listing 1: Early Encapsulation Implementation

Encapsulation refers to grouping data (attributes) and methods (functions) within a class. For instance, the fields `name` and `universityId` are defined as `protected` attributes, allowing access from subclasses and same-package classes, while public getters (`getName` and `getUniversityId`) provide controlled external access. However, using `protected` weakens encapsulation, as subclasses can still modify internal state.

2.1.2 Inheritance

```
1 class WarwickStudent extends WarwickPerson {
2
3     public WarwickStudent(String name, String studentId) {
4         super(name, studentId);
5     }
6 }
7
8 class WarwickStaff extends WarwickPerson {
9
10    public WarwickStaff(String name, String staffId) {
11        super(name, staffId);
12    }
13 }
```

Listing 2: Early Inheritance implementation

Inheritance refers to classes reusing attributes and behaviours from other classes, forming hierarchical relationships. For example, both `WarwickStudent` and `WarwickStaff` inherit attributes and behaviours from their superclass (`WarwickPerson`), encouraging code reuse and development.

2.1.3 Polymorphism

```
1 public class Main {
2     public static void main(String[] args) {
3
4         WarwickStudent s1 = new WarwickStudent("Yasir", "u5653630");
5         WarwickStaff s2 = new WarwickStaff("Dr Smith", "u2193684");
6
7         printRole(s1);
8         printRole(s2);
9     }
10
11    public static void printRole(WarwickPerson p) {
12        System.out.println(p.getRole() + ": " + p.getName());
13    }
14 }
15
16 class WarwickPerson {
17     protected String name;
18     protected String universityId;
19
20    public WarwickPerson(String name, String universityId) {
21        this.name = name;
22        this.universityId = universityId;
23    }
24
25    public String getName() { return name; }
26 }
```

```

27     public String getRole() {
28         return "Warwick Person";
29     }
30 }
31
32 class WarwickStudent extends WarwickPerson {
33     public WarwickStudent(String name, String studentId) {
34         super(name, studentId);
35     }
36
37     @Override
38     public String getRole() {
39         return "Student";
40     }
41 }
42
43 class WarwickStaff extends WarwickPerson {
44     public WarwickStaff(String name, String staffId) {
45         super(name, staffId);
46     }
47
48     @Override
49     public String getRole() {
50         return "Staff";
51     }
52 }

```

Listing 3: Early Polymorphism implementation

Polymorphism refers to objects sharing a common interface to exhibit different forms of behaviour depending on their instantiated type. For example, `WarwickStaff` and `WarwickStudent` instances share a common superclass type (`WarwickPerson`) during runtime, where the appropriate subclass implementation of `getRole()` is executed based on the instantiated object. This reduces duplication through a single method, improving flexibility and maintainability.

2.2 OOP Security Consideration

Even though the code is functional and able to use some of the core OOP principles, such as inheritance (`WarwickStudent extends WarwickPerson`) and polymorphism (overriding `getRole()` in subclasses), their effects are severely diluted. This is because encapsulation is weakened by using `protected` fields instead of `private` ones, which enables subclasses to directly access/modify state (e.g., `name`, `universityId`) without controlled access. Furthermore, the behavioural responsibility of the objects is reduced by using the `getRole()` method to return a string value, since this shifts the output logic into the `Main` class via the `printRole()` function. These flaws introduced security weaknesses such as improper encapsulation and state manipulation, which I was unaware of. These weaknesses were later addressed in Section 3.1.

2.3 Advantages of OOP (Java)

- The principle of data hiding helps the programmer to build secure programs which cannot be modified by the code in other parts of the program.
- OOP languages such as Java allow developers to break the program into the bit-sized problems that can be solved easily (one object at a time)
- Highly modular and allows code reuse from small to large systems. This is particularly relevant in industry, where Java powers 90% of Fortune 500 companies codebases [3].
- It is very easy to partition the work in a project based on objects.

2.4 Limitations of OOP (Java)

- OOP languages such as Java take time to get used to, with a steeper learning curve. In contrast, existing languages such as Python offer the same OOP principles and benefits, but using minimalistic syntax and extensive standard library, which in turn supports rapid development cycle and high productivity for beginners [4].
- OOP cannot be used everywhere as it is not a universal solution. Some problems, such as those requiring real-time event handling or asynchronous GUI interactions, require a more event-driven approach.
- Not suitable for small projects, since it requires a well-thought-through design
- Difficult to make efficient and flexible classes for optimum reusability

2.5 Alternative Implementation to OOP

Although the code is functional and able to achieve its goal, the same solution can be implemented using Python, as seen below:

```
1 class WarwickPerson:
2     def __init__(self, name, university_id):
3         # Leading underscore = "internal attribute" by convention
4         .
5         self._name = name
6         self._university_id = university_id
7
8     def display_role(self):
9         print(f"Warwick Person: {self._name}")
10
11 class WarwickStudent(WarwickPerson):
12     def display_role(self):
13         print(f"Student: {self._name}")
14
15 class WarwickStaff(WarwickPerson):
16     def display_role(self):
17         print(f"Staff: {self._name}")
```

```

18 def main():
19     # Polymorphism: base-type usage via a shared interface (
        display_role)
20     people = [
21         WarwickStudent("Yasir", "u5653630"),
22         WarwickStaff("Dr Smith", "u2193684"),
23     ]
24
25     for p in people:
26         p.display_role()
27
28 if __name__ == "__main__":
29     main()

```

Listing 4: Python UIS hierarchy using naming conventions for internal attributes

This would have been an easier option since Python uses naming conventions (e.g, variable, method) to reduce syntactic complexity by not having to explicitly declare private or public access modifiers, instead relying on naming conventions such as `_protected` and `__private`, shifting boundary enforcement responsibility from the compiler to the developer.

2.6 Event-Driven Programming (EDP)

EDP is a programming paradigm that enables program execution to be determined by events such as user actions or messages [5]. Its main aim is to facilitate highly reactive, responsive, and user-friendly GUI applications built using languages such as Java, JavaScript and Python [6].

While having my first attempt solving exercise 2 of the event handling lab, I naively prioritised responsiveness to user input instead of analysing the broader design in my early solution (Appendix B), with their individual parts presented below:

2.6.1 Event Loop

```
1 public class SimpleProductApp_Early {
2
3     static ArrayList<String> cart = new ArrayList<>();
4
5     public static void main(String[] args) {
6
7         JFrame frame = new JFrame("Simple Product App");
8         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9
10        JPanel productPanel = new JPanel();
11        productPanel.setLayout(new GridLayout(3, 1));
12
13        frame.setLayout(new BorderLayout());
14        frame.add(productPanel, BorderLayout.CENTER);
15
16        frame.setSize(300, 150);
17        frame.setVisible(true); // starts UI processing (EDT
                                // event loop)
18    }
19 }
```

Listing 5: Early Event Loop / UI Bootstrapping Implementation

Event loops are continuous processes that wait for and dispatch events in a program. For instance, in the code above the loop is implicitly activated when calling `frame.setVisible(true)`, transferring control to Swing's Event Dispatch Thread (EDT). Prior to this step, the layout manager is used to configure the layout settings and adding the `productPanel` to the frame, ensuring all UI components are correctly registered before event processing begins.

2.6.2 Event Sources

```
1 String[] products = {"Product 1", "Product 2", "Product 3"};
2
3 for (int i = 0; i < products.length; i++) {
4     JButton button = new JButton(products[i]); // event source
5     productPanel.add(button);                 // source added
6     to UI
7 }
```

Listing 6: Early Event Sources Implementation

These are the origins of events, such as user interactions (e.g., button clicks) or system-generated signals, and they are emitted by the `JButton` event source. This acts as an event source to create new UI components for each product ("Product 1", "Product 2", "Product 3"), so that an `ActionEvent` is generated.

2.6.3 Event Listeners

```
1 button.addActionListener(new ActionListener() {
2     @Override
3     public void actionPerformed(ActionEvent e) {
4         // handler body shown in next listing
5     }
6 });
```

Listing 7: Early Event Listener Registration

Event listeners refer to components responsible for monitoring event sources and wait for specific triggers (e.g, button click events).

2.6.4 Event Handlers

```
1 @Override
2 public void actionPerformed(ActionEvent e) {
3
4     JButton clicked = (JButton) e.getSource();
5     String selectedName = clicked.getText();
6
7     cart.add(selectedName);
8
9     System.out.println(selectedName + " has been added to cart");
10    System.out.println("Cart now has " + cart.size() + " items");
11 }
```

Listing 8: Early Event Handler Implementation

These refer to functions or methods that execute when an event occurs. For instance, `actionPerformed()` responds to button clicks, and handles the functionality intended, involving adding an item to the cart and then displaying the total amount of cart items present, with the operation `println`.

2.7 EDP Security Consideration

Even though this initial solution turned out to be functional, the event-driven structure resulted in decentralised state mutation, where each `ActionListener` independently modified the application state, preventing any separation of logic between the presentation and logic layers. Furthermore, the `cart` was declared as being static and globally accessible, introducing a shared mutable state without access control, as the lack of validation before state mutation allowed payloads to be implicitly trusted. By prioritising functionality without any clear trust boundaries, the system developed several structural weaknesses, since it became harder to modify cart behaviour without editing every handler. This is because it became harder to test logic independently of the GUI, increasing the chance of inconsistent validation as the system grows without defensive programming around input. The revised solution to these weaknesses were later shown in Section 3.2.

2.8 Advantages & limitations of EDP (Java)

Solutions implemented using EDP benefit from asynchronous processing, which improves the responsiveness of programs and supports future scalability by efficiently handling larger number of concurrent operations [7]. This is particularly the case with languages like Java, which provide structured support for event-driven programming, since they offer loose coupling, and support the creation of more modular, flexible and maintainable code. However, this paradigm also comes with its own set of challenges, with the main one being increased code complexity. This is particularly the case for syntax heavy languages like java, where a growing codebase makes the process of testing and debugging harder, thereby increasing the likelihood of developers accidentally introducing architectural security risks, including race conditions [8].

2.9 Alternative Implementation to EDP

EDP implementation could have also been changed to Python, since it comes with pre-built GUI building libraries, such as Tkinter. This reduces the need to explicitly implement listener interfaces as required in Java's Swing framework, simplifying the process of defining structural boundaries (event dispatch mechanisms), preventing control flow abuse and improving architectural separation.

```
1 import tkinter as tk
2
3 def handle_click():
4     print("Button clicked")
5
6 root = tk.Tk()
7 root.title("Simple GUI")
8
9 # Event binding via direct callback (no explicit listener
   interface)
10 button = tk.Button(root, text="Click Me", command=handle_click)
11 button.pack()
12
13 root.mainloop()
```

Listing 9: Simple Tkinter GUI with callback binding

2.10 Functional Programming (FP)

Functional programming is an approach to software development in which applications are constructed using either pure mathematical functions or recursion to achieve a desired outcome [9]. This type of programming is normally written with Haskell, C# and C++, but they can also be supported by other languages such as Java.

My initial understanding of functional programming was limited to simple deterministic functions such as squaring a number, as shown in Figure 1.

$$\begin{aligned} f &: \mathbb{Z} \rightarrow \mathbb{Z} \\ f(x) &= x^2 \\ \forall x \in \mathbb{Z}, f(x) &= x \cdot x \end{aligned}$$

Figure 1: A deterministic pure function mapping an integer to its square

To apply this initial understanding in practice, I produced a small functional-style implementation to solve quadratics (Appendix C).

2.11 FP Security Consideration

Although the local implementation in Appendix C managed to meet all the necessary functional requirements, it still left several residual risks that can be exploited by attackers if the program is exposed as a service (API / web app). Firstly, the program does not check for input size, since the `read()` operation accepts arbitrarily long strings from `Scanner.nextLine()`. This means attackers could send many large requests rapidly, with each request consuming memory and CPU, eventually leading to heap exhaustion and server crash (CWE-400). Furthermore, the program performs silent error handling and hides bad states, with the `catch (Exception e) return null;` operation, which swallows all parse errors and makes it harder to detect malicious/abusive input patterns (CWE-703). Despite these residual risks, mitigation strategies were later introduced and discussed in Section 3.3

2.12 Advantages & limitations of FP (Java)

This paradigm makes use of deterministic functions to ensure predictable execution by making parallelism easy to handle, facilitating the debugging process. This in turn allows for safer concurrent execution by not sharing data structures, enhancing code readability and maintainability benefits. However, this type of paradigm has a steep learning curve as developers are required to use an alternative style of thinking, particularly when using sensitive topics such as recursion to optimise system performance. Moreover, this paradigm is not very suited for Java, since Java is more of an OOP language, but this paradigm requires a language more oriented with FP principles, such as Haskell.

2.13 Alternative Implementation of FP (Haskell)

```

1 import Text.Printf
2 eps=1e-12
3 main=do
4   xs<-fmap (map read . words) getContents; let [a,b,c]=xs
5   let sol | abs a<eps = if abs b<eps then if abs c<eps then ("
      Infinite solutions (0=0).",[[]]) else ("No solution (c!=0).",
      ,[]) else ("Linear (a=0).",[[-c/b]])
6       | d<-b*b-4*a*c, d<-d = if d< -eps then ("No real roots.
      ",[[]]) else if abs d<=eps then ("One real root (
      double).",[[-b/(2*a)]) else let s=sqrt d; r1=(-b-s)
      /(2*a); r2=(-b+s)/(2*a) in ("Two real roots.",if r1
      <=r2 then [r1,r2] else [r2,r1])
7   let (m,rs)=sol
8   putStrLn m
9   if null rs then pure () else printf "roots: %s\n" (show rs)

```

Listing 10: Haskell quadratic solver implementation

3 Reflection

This section outlines my growth and understanding of the principles behind OOP, EDP, Reactive and FP paradigms. It highlights the security issues in my earlier implementations, lessons learned, and their impact on my revised secure solutions.

3.1 Revised OOP Solution

After highlighting the issues found in my early UIS implementation (Section 2.2), I decided to overcome these constraints by revisiting the existing implementation using a requirements-driven design approach. To support my planning, I made use of a class diagram (Figure 2), in order to provide a clear visual representation of the system's structure (classes, attributes, relationships). The complete revised implementation is presented in Appendix D, with its core principles outlined below.

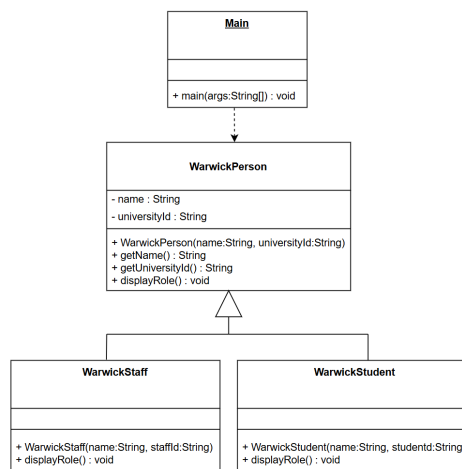


Figure 2: UML Class Diagram for the WarwickPerson hierarchy

3.1.1 Revised Encapsulation

```
1 class WarwickPerson {
2
3     private String name;
4     private String universityId;
5
6     public WarwickPerson(String name, String universityId) {
7         this.name = name;
8         this.universityId = universityId;
9     }
10
11     public String getName() {
12         return name;
13     }
14
15     public String getUniversityId() {
16         return universityId;
17     }
18 }
```

Listing 11: Revised Encapsulation Implementation

3.1.2 Revised Inheritance

```
1 class WarwickStudent extends WarwickPerson {
2
3     public WarwickStudent(String name, String studentId) {
4         super(name, studentId);
5     }
6 }
7
8 class WarwickStaff extends WarwickPerson {
9
10     public WarwickStaff(String name, String staffId) {
11         super(name, staffId);
12     }
13 }
```

Listing 12: Revised Inheritance Implementation

3.1.3 Revised Polymorphism

```
1 public class Main {
2     public static void main(String[] args) {
3
4         WarwickPerson s1 = new WarwickStudent("Yasir", "u5653630"
5         );
6         WarwickPerson s2 = new WarwickStaff("Dr Smith", "u2193684
7         ");
8
9         s1.displayRole();
10        s2.displayRole();
11    }
```

```

9      }
10   }
11
12   class WarwickPerson {
13
14       private String name;
15       private String universityId;
16
17       public WarwickPerson(String name, String universityId) {
18           this.name = name;
19           this.universityId = universityId;
20       }
21
22       public String getName() {
23           return name;
24       }
25
26       public String getUniversityId() {
27           return universityId;
28       }
29
30       public void displayRole() {
31           System.out.println("Warwick Person: " + name);
32       }
33   }
34
35   class WarwickStudent extends WarwickPerson {
36
37       public WarwickStudent(String name, String studentId) {
38           super(name, studentId);
39       }
40
41       @Override
42       public void displayRole() {
43           System.out.println("Student: " + getName());
44       }
45   }
46
47   class WarwickStaff extends WarwickPerson {
48
49       public WarwickStaff(String name, String staffId) {
50           super(name, staffId);
51       }
52
53       @Override
54       public void displayRole() {
55           System.out.println("Staff: " + getName());
56       }
57   }

```

Listing 13: Revised Polymorphism Implementation

This approach enabled me to realise how responsibility should be clearly divided between different roles (e.g., `Main`, `WarwickPerson`, `WarwickStudent`, `WarwickStaff`), in order to support separation of concerns. Moreover, attributes such as `name` and `universityId` were labelled to be `private` over `protected`, ensuring subclasses were not able to directly modify identity fields. These reflections aided the revised solution (Appendix D), reducing the likelihood of improper access control and unintended state modification.

This way inheritance aligns with the hierarchical relationship roles. However, as the code-base grows with more components (roles/features/trust boundary concerns), composition could be used to provide flexibility in large systems by reducing tight coupling between roles and identity structures.

3.2 Revised EDP Solution

Having examined the limitations for my early exercise 2 implementation in Section 2.7, I created a new solution (Appendix E) that removed the global static shared state, enabling business logic to be separated from the GUI layer and state mutation to be centralised within a dedicated `Cart` class. These updates strengthened encapsulation with clearer responsibility boundaries, improving testability independent of the interface, reducing implicit trust in event payloads.

3.2.1 Revised Event Loop

```
1 JFrame frame = new JFrame("Simple Product App");
2 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
3
4 JPanel productPanel = new JPanel();
5 productPanel.setLayout(new GridLayout(3, 1));
6
7 ProductSelectionEventDispatcher dispatcher =
8     new ProductSelectionEventDispatcher();
9
10 Cart cart = new Cart();
```

Listing 14: Revised Event Loop / UI Bootstrapping

3.2.2 Revised Event Sources

```
1 Product[] products = {
2     new Product("Product 1"),
3     new Product("Product 2"),
4     new Product("Product 3")
5 };
6
7 for (Product product : products) {
8     JButton button = new JButton(product.getName());
```

Listing 15: Revised Event Sources Using Controlled Product Objects

3.2.3 Revised Event Listeners

```
1 ProductSelectionListener listener =
2     new ProductSelectionListener() {
3         @Override
4         public void productSelected(Product selectedProduct) {
5
6             if (selectedProduct != null) {
7                 cart.add(selectedProduct);
8                 System.out.println(
9                     selectedProduct.getName() + " has been added
10                        to cart"
11                );
12                 System.out.println(
13                     "Cart now has " + cart.size() + " items"
14                );
15             }
16         };
17
18 dispatcher.addProductSelectionListener(listener);
```

Listing 16: Revised Event Listener with Centralised Business Logic

3.3 Revised Future FP Solution

To address the weaknesses mentioned in Section 2.7, the program should enforce strict input length and numeric range constraints to prevent uncontrolled resource consumption (CWE-400), alongside request throttling if deployed as a network-facing service. Additionally, exception handling should be refined by catching specific parsing errors and logging failures rather than silently returning null, thereby improving observability and reducing improper handling of exceptional conditions (CWE-703).

4 Conclusion

This report documents my progression in secure software development within the PLCS module.

References

- [1] Rishabh Prabhu Srivastava. *Object Oriented Programming (OOP) Concepts in Java*. <https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>. Last Updated: 23 Jan, 2026. 2026.
- [2] Alexander S. Gillis and Sarah Lewis. *Object-oriented programming (OOP)*. <https://www.techtarget.com/searchapparchitecture/definition/object-oriented-programming-OOP>. Published: 14 June 2024. 2024.
- [3] Marc Haberkorn. *Java at 30 and Beyond: How IBM Is Securing the Future of Java*. <https://www.ibm.com/new/announcements/java-at-30-and-beyond-how-ibm-is-securing-the-future-of-java>. Published: 02 October 2025. IBM Announcement. 2025.
- [4] Mariana Berga and Rute Figueiredo. *Python vs Java: Key Differences, Performance, and Use Cases*. <https://www.imaginarycloud.com/blog/python-vs-java>. Published April 15, 2025. 2025.
- [5] Amit Kumar Kohli. *What is the Event-Driven Programming Paradigm?* <https://www.geeksforgeeks.org/javascript/what-is-the-event-driven-programming-paradigm/>. Last Updated: 23 Jul, 2025. 2025.
- [6] William Stoll. *The Power of Event-Driven Programming Language: A Comprehensive Guide*. <https://techidemics.com/the-power-of-event-driven-programming-language-a-comprehensive-guide/>. Accessed: 2026-02-15. n.d.
- [7] Convex. *Event-Driven Programming*. <https://stack.convex.dev/event-driven-programming>. Published: 2 years ago. 2024.
- [8] Algodcademy. *Introduction to Event-Driven Programming: A Comprehensive Guide*. <https://algocademy.com/blog/introduction-to-event-driven-programming-a-comprehensive-guide/>. Accessed: 2026-02-15. n.d.
- [9] Bob Reselman. *Understanding the Principles of Functional Programming*. <https://www.theserverside.com/tip/Understanding-the-principles-of-functional-programming>. Published: 27 Nov 2024. 2024.

Appendices

A Java Implementation of OOP Principles

```
1 public class Main {
2     public static void main(String[] args) {
3
4         WarwickStudent s1 = new WarwickStudent("Yasir", "u5653630
5         ");
6         WarwickStaff s2 = new WarwickStaff("Dr Smith", "u2193684"
7         );
8
9         printRole(s1);
10        printRole(s2);
11    }
12
13    public static void printRole(WarwickPerson p) {
14        System.out.println(p.getRole() + ": " + p.getName());
15    }
16 }
17
18 class WarwickPerson {
19
20     protected String name;
21     protected String universityId;
22
23     public WarwickPerson(String name, String universityId) {
24         this.name = name;
25         this.universityId = universityId;
26     }
27
28     public String getName() {
29         return name;
30     }
31
32     public String getUniversityId() {
33         return universityId;
34     }
35
36     public String getRole() {
37         return "Warwick Person";
38     }
39 }
40
41 class WarwickStudent extends WarwickPerson {
42
43     public WarwickStudent(String name, String studentId) {
44         super(name, studentId);
45     }
46
47     @Override
```

```
46     public String getRole() {
47         return "Student";
48     }
49 }
50
51 class WarwickStaff extends WarwickPerson {
52
53     public WarwickStaff(String name, String staffId) {
54         super(name, staffId);
55     }
56
57     @Override
58     public String getRole() {
59         return "Staff";
60     }
61 }
```

Listing 17: Early OOP implementation

B Java Implementation of EDP Principles

```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 import java.util.ArrayList;
6
7 public class SimpleProductApp_Early {
8
9     // Store selected products in a simple cart
10    static ArrayList<String> cart = new ArrayList<>();
11
12    public static void main(String[] args) {
13
14        // Create main window
15        JFrame frame = new JFrame("Simple Product App");
16        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17
18        // Panel to hold product buttons
19        JPanel productPanel = new JPanel();
20        productPanel.setLayout(new GridLayout(3, 1));
21
22        // Basic product list
23        String[] products = { "Product 1", "Product 2", "Product
24                               3" };
25
26        // Create a button for each product
27        for (int i = 0; i < products.length; i++) {
28
29            JButton button = new JButton(products[i]);
30
31            // Handle button click
32            button.addActionListener(new ActionListener() {
33                @Override
34                public void actionPerformed(ActionEvent e) {
35
36                    // Get button that was clicked
37                    JButton clicked = (JButton) e.getSource();
38                    String selectedName = clicked.getText();
39
40                    // Add selected product to cart
41                    cart.add(selectedName);
42
43                    // Display confirmation
44                    System.out.println(selectedName + " has been
45                                   added to cart");
46                    System.out.println("Cart now has " + cart.
47                                   size() + " items");
48                }
49            });
50        }
51    }
52 }
```

```
47         productPanel.add(button);
48     }
49
50
51     // Add panel to frame
52     frame.setLayout(new BorderLayout());
53     frame.add(productPanel, BorderLayout.CENTER);
54
55     frame.setSize(300, 150);
56     frame.setVisible(true);
57 }
58 }
```

Listing 18: Initial event-driven implementation of SimpleProductApp

C Java Implementation of FP Principles

```
1 import java.util.*;
2
3 public class QuadFPMIn {
4
5     // ----- Immutable data -----
6     static final double EPS = 1e-12;
7     record Sol(String msg, List<Double> roots) {}
8
9     // ----- Pure function -----
10    static Sol solve(double a, double b, double c) {
11
12        // ----- Declarative logic -----
13        if (Math.abs(a) < EPS) {
14            if (Math.abs(b) < EPS)
15                return Math.abs(c) < EPS
16                    ? new Sol("Infinite solutions (0=0).", List.of())
17                    : new Sol("No solution (c!=0).", List.of());
18
19            return new Sol("Linear (a=0).", List.of(-c / b));
20        }
21
22        // ----- Deterministic function -----
23        double d = b * b - 4 * a * c;
24
25        // ----- Declarative logic -----
26        if (d < -EPS)
27            return new Sol("No real roots (d<0). d=" + d, List.of());
28
29        if (Math.abs(d) <= EPS)
30            return new Sol("One real root (double).", List.of(-b / (2 *
31                a)));
32
33        double s = Math.sqrt(d);
34        double x1 = (-b - s) / (2 * a);
35        double x2 = (-b + s) / (2 * a);
36
37        // ----- Immutable data -----
38        return new Sol("Two real roots.",
39            x1 <= x2 ? List.of(x1, x2) : List.of(x2, x1));
40    }
41
42    // ----- Input -----
43    static Double read(Scanner sc, String p) {
44        System.out.print(p);
45        String t = sc.nextLine().trim();
46        try {
47            double v = Double.parseDouble(t);
48            return (Double.isFinite(v) ? v : null);
49        } catch (Exception e) {}
50    }
51 }
```

```

49     return null;
50 }
51 }
52
53 // ----- Program Entry -----
54 public static void main(String[] args) {
55
56     try (Scanner sc = new Scanner(System.in)) {
57
58         Double a = read(sc, "a: ");
59         Double b = read(sc, "b: ");
60         Double c = read(sc, "c: ");
61
62         // ----- Declarative logic -----
63         if (a == null || b == null || c == null) {
64             System.out.println("Error: enter valid numbers for a,b,c.
65                                 ");
66             return;
67         }
68
69         // ----- Pure function call -----
70         Sol sol = solve(a, b, c);
71
72         // ----- Output -----
73         System.out.println(sol.msg);
74         if (!sol.roots.isEmpty())
75             System.out.println("roots: " + sol.roots);
76     }
77 }

```

Listing 19: Functional-style quadratic solver demonstrating pure computation, immutability, and declarative logic

D Revised UIS class hierarchy

```
1 public class Main {
2     public static void main(String[] args) {
3
4         // Polymorphism: parent reference holding child objects
5         WarwickPerson s1 = new WarwickStudent("Yasir", "u5653630"
6         );
7         WarwickPerson s2 = new WarwickStaff("Dr Smith", "u2193684
8         ");
9
10        // Runtime method dispatch (polymorphism)
11        s1.displayRole();
12        s2.displayRole();
13    }
14 }
15
16 class WarwickPerson {
17
18     private String name;
19     private String universityId;
20
21     public WarwickPerson(String name, String universityId) {
22         this.name = name;
23         this.universityId = universityId;
24     }
25
26     public String getName() {
27         return name;
28     }
29
30     public String getUniversityId() {
31         return universityId;
32     }
33
34     public void displayRole() {
35         System.out.println("Warwick Person: " + name);
36     }
37 }
38
39 class WarwickStudent extends WarwickPerson {
40
41     public WarwickStudent(String name, String studentId) {
42         super(name, studentId);
43     }
44
45     @Override
46     public void displayRole() {
47         System.out.println("Student: " + getName());
48     }
49 }
```

```
48
49 class WarwickStaff extends WarwickPerson {
50
51     public WarwickStaff(String name, String staffId) {
52         super(name, staffId);
53     }
54
55     @Override
56     public void displayRole() {
57         System.out.println("Staff: " + getName());
58     }
59 }
```

Listing 20: Revised UIS class hierarchy

E Revised event-driven implementation

```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 import java.util.ArrayList;
6 import java.util.EventListener;
7 import java.util.EventObject;
8 import javax.swing.event.EventListenerList;
9
10 public class SimpleProductApp_Revised {
11
12     public static void main(String[] args) {
13
14         // Create main window
15         JFrame frame = new JFrame("Simple Product App");
16         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17
18         // Panel for product buttons
19         JPanel productPanel = new JPanel();
20         productPanel.setLayout(new GridLayout(3, 1));
21
22         // Controlled product definitions
23         Product[] products = {
24             new Product("Product 1"),
25             new Product("Product 2"),
26             new Product("Product 3")
27         };
28
29         // Central event dispatcher
30         ProductSelectionEventDispatcher dispatcher =
31             new ProductSelectionEventDispatcher();
32
33         // Encapsulated cart instance (no global static access)
34         Cart cart = new Cart();
35
36         // Listener responsible for business logic
37         ProductSelectionListener listener = new
38             ProductSelectionListener() {
39             @Override
40             public void productSelected(Product selectedProduct)
41             {
42                 // Defensive check before state mutation
43                 if (selectedProduct != null) {
44                     cart.add(selectedProduct);
45                     System.out.println(selectedProduct.getName()
46                         + " has been added to cart");
47                     System.out.println("Cart now has "
48                         + cart.size() + " items");
```

```

48         }
49     }
50 };
51
52 dispatcher.addProductSelectionListener(listener);
53
54 // UI layer only triggers events
55 for (Product product : products) {
56     JButton button = new JButton(product.getName());
57
58     button.addActionListener(new ActionListener() {
59         @Override
60         public void actionPerformed(ActionEvent e) {
61             dispatcher.fireProductSelectionEvent(product)
62             ;
63         }
64     });
65
66     productPanel.add(button);
67 }
68
69 frame.setLayout(new BorderLayout());
70 frame.add(productPanel, BorderLayout.CENTER);
71
72 frame.setSize(300, 150);
73 frame.setVisible(true);
74 }
75
76 // Immutable product representation
77 static class Product {
78     private final String name;
79
80     public Product(String name) {
81         if (name == null || name.trim().isEmpty()) {
82             throw new IllegalArgumentException("Invalid
83             product name");
84         }
85         this.name = name;
86     }
87
88     public String getName() {
89         return name;
90     }
91 }
92
93 // Cart encapsulates state mutation
94 static class Cart {
95     private final ArrayList<Product> items = new ArrayList
96     <>();
97
98     public void add(Product product) {

```

```

96         items.add(product);
97     }
98
99     public int size() {
100         return items.size();
101     }
102 }
103
104 // Custom event object
105 static class ProductSelectionEvent extends EventObject {
106     private final Product selectedProduct;
107
108     public ProductSelectionEvent(Object source,
109                                 Product selectedProduct) {
110         super(source);
111         this.selectedProduct = selectedProduct;
112     }
113
114     public Product getSelectedProduct() {
115         return selectedProduct;
116     }
117 }
118
119 interface ProductSelectionListener extends EventListener {
120     void productSelected(Product selectedProduct);
121 }
122
123 // Dispatcher centralises event propagation
124 static class ProductSelectionEventDispatcher {
125
126     private final EventListenerList listenerList =
127         new EventListenerList();
128
129     public void addProductSelectionListener(
130         ProductSelectionListener listener) {
131         listenerList.add(ProductSelectionListener.class,
132             listener);
133     }
134
135     public void fireProductSelectionEvent(Product
136         selectedProduct) {
137
138         Object[] listeners = listenerList.getListenerList();
139
140         ProductSelectionEvent event =
141             new ProductSelectionEvent(this,
142                 selectedProduct);
143
144         for (int i = 0; i < listeners.length; i = i + 2) {
145             if (listeners[i] ==
146                 ProductSelectionListener.class) {

```

```

144         ((ProductSelectionListener)
145             listeners[i + 1])
146             .productSelected(
147                 event.getSelectedProduct());
148     }
149 }
150 }
151 }
152 }

```

Listing 21: Revised event-driven implementation with centralised state control

F Code Base Access

The complete source code for both the early and revised implementations, including all completed lab exercises referenced in this report, is available at the following repository:

Link: <https://git-lab.cyber.warwick.ac.uk/c5653630/PLCS-CW1>